

Python Programming

Week 1 - Python Fundamentals 1

Tuesday Lesson: Data Types, Strings, Operators, and Input

Session Tuesday	Duration 2 Hours	Level Beginner	Assignment Included
---------------------------	----------------------------	--------------------------	-------------------------------

This lesson introduces the core building blocks Python uses to represent information and perform useful work: data types, strings, operators, type conversion, and user input. By the end of the session, students should be able to identify common data types, format and manipulate text, perform calculations, and collect input from a user.

Lesson Overview

- Core data types: int, float, str, and bool.
- Working with strings using concatenation, f-strings, methods, indexing, slicing, and len().
- Using arithmetic operators, floor division, modulus, exponentiation, and operator precedence.
- Converting between types and collecting user input with input().
- Building a simple calculator and completing the Tuesday assignment.

Review From Monday

Before moving into new material, it is useful to reconnect Tuesday's lesson to Monday's foundations. On Monday, students learned what Python is, how Python reads and runs code, how comments work, and how variables point to values in memory.

The most important ideas to remember are:

- A variable is a name that points to a value.
- The # symbol begins a comment, which Python ignores when running code.
- The Python interpreter reads and executes code from top to bottom, one line at a time.
- Python stops when it reaches an error, so reading error messages calmly is an important programming skill.

1. Core Data Types: int, float, str, and bool

Every piece of information Python works with has a type. A type tells Python what kind of data something is and what operations are allowed on it. This matters because Python treats numbers, text, and True/False values differently.

For example, the value 30 can be used in mathematical calculations, but the text "30" is treated as a string because it is inside quotation marks. Python will not assume that text should be used as a number unless you explicitly convert it.

Types are one of the most common sources of beginner errors. Learning to recognize them early makes it easier to understand error messages and fix problems quickly.

Integers: int

An integer is a whole number with no decimal point. Integers can be positive, negative, or zero.

```
age = 30
employees = 1500
temperature = -5
score = 0
```

```
print(type(age)) # Output: <class 'int'>
```

The type() function shows the type of a value or variable. It is useful when you are unsure what kind of data your program is working with.

Floating-Point Numbers: float

A float is a number with a decimal point. Floats are used for measurements, prices, percentages, temperatures, and other values that may include fractional parts.

```
price = 29.99
height = 1.75
temperature = -3.5
percentage = 0.15
```

```
print(type(price)) # Output: <class 'float'>
```

Important float precision note

Floats can sometimes produce tiny precision surprises. For example, $0.1 + 0.2$ may display as 0.30000000000000004. This is not a Python bug. It happens because computers store decimal values in binary. For everyday beginner programs this usually does not matter, but financial software often uses Python's decimal module for more exact currency calculations.

Strings: str

A string is a sequence of characters. Strings are used for text, such as names, cities, job titles, messages, phone numbers, and IDs that should not be treated as numbers. A string is created by placing text inside single quotes or double quotes.

```
name = "Funmilayo"
city = 'Lagos'
job_title = "Software Developer"
empty_string = ""
```

```
print(type(name)) # Output: <class 'str'>
```

A value that looks like a number is still a string if it is written inside quotation marks.

```
phone_number = "08012345678" # This is a string
employee_id = 12345          # This is an integer
```

Phone numbers are usually stored as strings because you do not normally calculate with them, and leading zeros must be preserved.

Booleans: bool

A Boolean stores only one of two values: True or False. Booleans are used when code needs to make decisions, check conditions, or represent yes/no states.

```
is_logged_in = True
has_paid = False
is_over_18 = True
account_active = False
```

```
print(type(is_logged_in)) # Output: <class 'bool'>
```

In Python, True and False must begin with capital letters. Lowercase true and false are not valid Boolean values in Python.

A Simple Way to Think About Types

Data types are like sections on a form. A name field expects text, an age field expects a whole number, a salary field may expect a decimal number, and a checkbox expects a yes/no value. Python types work in a similar way. Each value has an expected kind of use, and mixing them incorrectly can cause errors.

Knowledge Check

1. Identify the type of each value without running code: 42, 3.14, "hello", True, "99", 0.
2. Explain why a phone number is often better stored as a string than as an integer.
3. Describe what the type() function does and when you would use it.

2. Working With Strings in Depth

Strings are one of the most important data types in programming because real-world data often arrives as text. Names, emails, addresses, form responses, CSV files, web requests, and API responses all involve strings. Being comfortable with strings is essential for web development, data work, automation, and backend programming.

String Concatenation and Repetition

The + operator joins strings together. This is called concatenation. Python does not automatically insert spaces, so spaces must be added manually where needed.

```
first_name = "Tunde"
last_name = "Adeyemi"
```

```
full_name = first_name + " " + last_name
print(full_name) # Output: Tunde Adeyemi
```

```
divider = "-" * 30
print(divider) # Output: -----
```

f-Strings

Although concatenation works, it can become messy when combining variables with text. A cleaner and more professional approach is to use f-strings. An f-string begins with the letter f before the opening quotation mark, and variables are placed inside curly braces.

```
name = "Chisom"
age = 28
city = "Abuja"

print(f"My name is {name} and I am {age} years old.")
print(f"{name} lives in {city}.")
print(f"In 10 years, {name} will be {age + 10} years old.")
```

f-strings are easier to read, easier to maintain, and reduce the need to manually convert numbers into strings when printing messages.

Common String Methods

Strings come with built-in tools called methods. A method is called by writing a dot after the string or variable, followed by the method name and parentheses.

```
name = " funmilayo adeyinka "

print(name.strip())      # Removes spaces from both ends
print(name.strip().upper()) # Converts to uppercase
print(name.strip().lower()) # Converts to lowercase
print(name.strip().title()) # Capitalizes each word

sentence = "I love Java"
print(sentence.replace("Java", "Python"))

csv_row = "Amara,30,Lagos"
parts = csv_row.split(",")
print(parts) # Output: ['Amara', '30', 'Lagos']
```

Methods such as `.strip()`, `.replace()`, and `.split()` are especially useful because real-world data is often messy. A name may include extra spaces, a sentence may need part of its text replaced, and a row from a spreadsheet or CSV file may need to be split into smaller pieces.

String Indexing

Every character in a string has a position number called an index. Python starts counting from 0. This means the first character is at position 0, the second character is at position 1, and so on.

```
language = "Python"

print(language[0]) # Output: P
print(language[1]) # Output: y
print(language[5]) # Output: n

print(language[-1]) # Output: n
print(language[-2]) # Output: o
```

Negative indexes count from the end of the string. `language[-1]` gives the last character.

String Slicing

Slicing allows you to extract part of a string. The pattern is [start:stop]. The start index is included, but the stop index is not included.

```
language = "Python"

print(language[0:3]) # Output: Pyt
print(language[2:5]) # Output: tho
print(language[:3]) # Output: Pyt
print(language[3:]) # Output: hon
```

Slicing is useful for extracting portions of text, such as the first few characters of a code, part of a date, or a first name from a longer string.

String Length With len()

The len() function returns the number of characters in a string. It counts letters, spaces, punctuation, and symbols.

```
print(len("Python"))      # Output: 6
print(len("Hello World")) # Output: 11
print(len(""))            # Output: 0

password = "mypassword123"
print(f'Your password is {len(password)} characters long.')
```

Practice Activity: Strings

4. Create a file called string_practice.py.
5. Store your full name in a variable. Use .strip() and .title() to clean and format it, then print it.
6. Use an f-string to print a sentence that introduces your name, previous profession, and years in that career.
7. Use string indexing or .split() to extract and print your first name.
8. Use .replace() on the sentence "Python is hard to learn" and replace "hard" with "rewarding".
9. Use len() to print how many characters are in your full name, including spaces.

Knowledge Check

10. If name = "Blessing", what does name[0] return? What does name[-1] return?
11. What is the difference between name.upper() and name = name.upper()?
12. Why are f-strings usually better than joining strings with +?
13. If you have the string "Lagos, Nigeria", how could you extract only "Nigeria"?

3. Operators and Expressions

Operators are symbols that perform actions on values. Python supports the standard arithmetic operators used in mathematics, along with a few operators that are especially useful in programming.

Arithmetic Operators

```
a = 17
b = 5

print(a + b) # Addition: 22
print(a - b) # Subtraction: 12
print(a * b) # Multiplication: 85
print(a / b) # Division: 3.4
print(a // b) # Floor division: 3
print(a % b) # Modulus: 2
print(a ** b) # Exponentiation: 1419857
```

The `/` operator performs normal division and always returns a float. The `//` operator performs floor division and gives the whole-number result. The `%` operator gives the remainder after division. The `**` operator raises a number to a power.

Floor Division and Modulus

Floor division is useful when you need whole units. For example, 130 minutes contains 2 full hours because `130 // 60` equals 2. The remaining minutes can be found with `130 % 60`, which equals 10.

```
total_minutes = 130
hours = total_minutes // 60
minutes_left = total_minutes % 60
```

```
print(hours) # Output: 2
print(minutes_left) # Output: 10
```

Modulus is also commonly used to check whether a number is even or odd. If `number % 2` equals 0, the number is even.

```
number = 24
print(number % 2) # Output: 0, so the number is even
```

Operator Precedence

When an expression contains multiple operators, Python follows a specific order of operations. Parentheses happen first, then exponentiation, then multiplication/division/floor division/modulus, then addition/subtraction.

```
result = 2 + 3 * 4
print(result) # Output: 14
```

```
result = (2 + 3) * 4
print(result) # Output: 20
```

When in doubt, use parentheses. They make your intention clear to Python and to other people reading your code.

Augmented Assignment Operators

Augmented assignment operators are shortcuts for updating the value of a variable. They make code shorter and easier to read.

```
score = 100

score += 10 # Same as score = score + 10
score -= 5  # Same as score = score - 5
score *= 2  # Same as score = score * 2
score //= 3 # Same as score = score // 3

print(score) # Output: 70
```

Practice Activity: Operators

14. Calculate how many full weeks are in 365 days using //.
15. Calculate how many days are left over after those full weeks using %.
16. A product costs 4500 Naira. Calculate the price after a 15% discount.
17. Predict the result of $2 + 3 ** 2 * 4 - 1$ before running it. Then verify your answer.

Knowledge Check

18. What is $25 \% 4$?
19. What is the difference between / and //?
20. If price = 200 and then price += 50, what is price? What if price *= 2 after that?

4. Type Conversion and User Input

Programs often receive data in one type and need to use it as another type. For example, a number typed by a user is received as text, but mathematical calculations require an integer or float. Type conversion is the process of changing a value from one type to another.

Converting Between Types

```
int("42")    # 42
int(3.9)     # 3
int(True)    # 1
int(False)   # 0

float("3.14") # 3.14
float(5)     # 5.0

str(42)      # '42'
str(3.14)    # '3.14'
str(True)    # 'True'

bool(1)      # True
bool(0)      # False
bool("")     # False
bool("hello") # True
```

Important conversion rule

int() truncates. It does not round. int(3.9) becomes 3, not 4. If you need to round to the nearest whole number, use round().

When Conversion Fails

Not every conversion is possible. If Python cannot logically convert a value into the requested type, it raises a `ValueError`.

```
int("42")    # Works
int("hello")  # ValueError
int("3.14")   # ValueError
```

```
int(float("3.14")) # Converts to float first, then to int: 3
```

Reading error messages calmly is part of becoming a programmer. Errors are not failures. They are information about what Python could not understand or complete.

Getting User Input

The `input()` function pauses the program and waits for the user to type something. Whatever the user types is stored as a string.

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

The most important rule is that `input()` always returns a string. Even if the user types 25, Python receives it as the string "25", not the integer 25.

```
# This causes an error because age is a string
age = input("How old are you? ")
age_in_ten_years = age + 10
```

```
# This works because the input is converted to an integer
age = int(input("How old are you? "))
age_in_ten_years = age + 10
print(f"In 10 years you will be {age_in_ten_years}.")
```

A helpful way to remember this is to imagine `input()` as a receptionist who writes down every response as text. If the response needs to be used in a calculation, the program must convert that text into a number first.

Building a Simple Calculator

The following calculator brings together user input, type conversion, operators, variables, and f-strings.

```
print("=== Simple Calculator ===")

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

total = num1 + num2
product = num1 * num2
quotient = num1 / num2
remainder = num1 % num2

print(f"\n{num1} + {num2} = {total}")
print(f"{num1} x {num2} = {product}")
print(f"{num1} / {num2} = {quotient:.2f}")
print(f"{num1} % {num2} = {remainder}")
```


The `:.2f` inside the f-string formats the number to two decimal places when printing. This is useful when displaying prices, percentages, or division results.

Practice Activity: Calculator

21. Run the calculator with 100 and 7.
22. Modify it to also print the floor division result using `//`.
23. Add a line that asks for the user's name and greets them before the calculation.
24. Test what happens when you type a word instead of a number. Read the error message carefully.

Knowledge Check

25. What does `input()` always return?
26. If a user types "25" and you want to add 5 to it, what must you do first?
27. What is the difference between `int(3.9)` and `round(3.9)`?
28. Why does the calculator use `float()` instead of `int()`?

Lesson Recap

This lesson introduced the four core data types in Python, how to work with strings, how arithmetic operators behave, and how to collect and convert user input. These concepts are foundational because almost every useful program receives data, stores it, transforms it, and produces output.

- `int` represents whole numbers.
- `float` represents decimal numbers.
- `str` represents text.
- `bool` represents True or False values.
- `type()` shows the type of a value or variable.
- String methods such as `.strip()`, `.upper()`, `.lower()`, `.replace()`, and `.split()` help clean and transform text.
- f-strings make it easy to combine variables and text.
- String indexing starts at 0, and slicing uses `[start:stop]`.
- `//` gives floor division, and `%` gives the remainder after division.
- `input()` always returns a string, so numbers from input must be converted before calculations.

Tuesday Assignment

Complete all tasks in one Python file called `week1_tuesday_assignment.py`. Add a comment at the top with your name and the date. The assignment is due before Thursday's class.

Part 1: Personal Profile App

Build a program that asks the user for the following information using `input()`:

- Full name
- Age as a number
- Previous job title
- Years spent in that previous career
- Monthly salary target in the new tech career as a number

After collecting the inputs, print a formatted career transition profile using f-strings. The output should look similar to this, using the user's actual data:

```
=== YOUR CAREER TRANSITION PROFILE ===  
Name:      Tunde Bakare  
Current age: 31  
Previous career: Accountant (8 years)  
New career goal: Software Developer  
Monthly target: N350,000  
Annual target:  N4,200,000  
Years to 40:  9
```

Part 2: String Operations

29. Take the user's full name and print it in three formats: ALL CAPS, all lowercase, and Title Case.
30. Count and print the number of characters in the full name, including spaces.
31. Extract and print just the first name. Hint: `.split()` makes this easier.

Part 3: Arithmetic Challenge

32. Ask the user to enter any number.
33. Calculate and print the square of that number using `**`.
34. Calculate and print the square root using `** 0.5`.
35. Use the modulus operator to determine whether the number is even or odd.
36. Calculate how many times 7 goes into the number and what the remainder is.

Submission Reminder

- Save the file as `week1_tuesday_assignment.py`.
- Add your name and the date in a comment at the top.
- Bring the file to Thursday's class ready to run.

Python Fundamentals - Week 1, Tuesday - Student Lesson Notes